A Role-Based Web Platform for the End-to-End Management of the Undergraduate Thesis and Graduation Process

Fernando Chantera, Harolt Farinango, Mateo Pilaquinga
School of Mathematical and Computational Sciences
Yachay Tech University
Urcuquí, Ecuador
{fernando.chantera, harolt.farinango, mateo.pilaquinga}@yachaytech.edu.ec

I. Introduction

Completing an undergraduate degree requires each student to pass through a long, multi-actor process: the registration and approval of a thesis topic, the appointment of a tutor and, optionally, a co-tutor, two consecutive thesis courses, a manuscript review by a committee, a similarity (plagiarism) check, the verification of administrative graduation requirements, and finally the scheduling and grading of an oral defense. In Ecuador, several of these requirements, such as pre-professional internships and community engagement (*vinculación con la sociedad*), are mandated by national regulation [1], so universities must not only run the process but also be able to prove that every requirement was met.

This document proposes the design and development of a web application that centralizes this process. The project continues the work of a previous Databases course, which produced a relational schema and an early prototype. The new system takes that inherited database as its starting point, corrects its design, and builds on top of it a secure, production-ready web application.

A. Problem Statement

At present, the graduation process is managed across disconnected tools: a separate system for official memos, an external tool for booking rooms, scattered administrative modules to validate student clearances, and local spreadsheets to coordinate calendars and committee assignments. Staff must consolidate this information by hand. As a result, delays are frequent, records become inconsistent, and there is no reliable trace of who approved what and when. Communication between students, tutors, reviewers, and coordinators also breaks down.

Fig. 1 summarizes the process that the system must support. Each stage depends on the previous one: for example, a defense cannot be scheduled until the committee is complete, the similarity check has been accepted, and every administrative requirement has been certified. When these dependencies are checked manually, a single missed step can postpone a student's graduation by an entire academic period.

The previous project left a valuable foundation, but it is not usable as-is. Our review found four mutually inconsistent versions of the database schema. In the schema actually used, critical business rules are not enforced: nothing prevents a thesis from having two main tutors, from having more than three reviewers, or from recording grades outside the valid range. The prototype also lacks authentication and role separation, audit history, room management, and the validation of administrative requirements before a defense. The problem this project addresses is therefore twofold: the institution needs a centralized system for the graduation process, and that system needs a sound, rule-enforcing data foundation to be trustworthy.

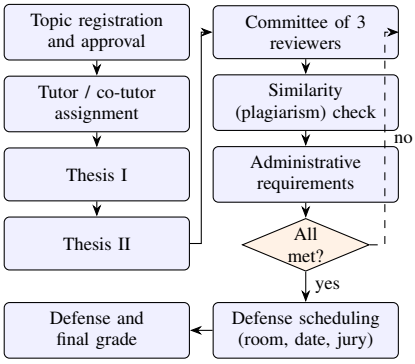


Fig. 1. Simplified undergraduate graduation workflow. Every stage depends on the previous ones, and the final gate requires the committee, the similarity check, and all administrative requirements to be complete.

B. Objectives

General objective: To design, develop, and deploy a secure, role-based web application that centralizes the management of the undergraduate thesis and graduation process, from topic registration to the final defense grade, using the corrected inherited database as its foundation.

Specific objectives:

1) To redesign the inherited relational schema so that the database itself enforces the main business rules: one main tutor and at most one co-tutor per thesis, at most three reviewers with exactly one committee president,

and valid grade and similarity ranges. Every change will be applied as a versioned migration.

2) To implement a RESTful API with authentication and role-based authorization for administrators, professors, and students. In this model, the tutor, reviewer, and president permissions are derived from each professor's assignments on each thesis, not from fixed roles.
3) To automate the workflow rules of Fig. 1: a thesis may advance only when its prerequisites are met, and a defense may be scheduled only after the committee, the similarity check, and the administrative requirements are complete, with no room or jury-member time conflicts.
4) To record a complete, auditable history of every thesis status change, including who made the change and when.
5) To build a responsive web interface tailored to each role, together with analytical reports on tutor workload, average defense grade by school, and defense-room availability.
6) To validate the system through automated and functional tests and to deploy it to a production environment.

C. Justification and Motivation

Relevance of the problem. The graduation process is the last and most administratively demanding step of a student's academic life. Errors here carry real costs: postponed defenses, repeated paperwork, and, in the worst case, a degree granted without a required certification. Because national regulation defines several of these requirements [1], the institution also needs evidence that each requirement was verified, which a spreadsheet-based process cannot reliably provide.

Usefulness of the proposed solution. A single platform replaces manual consolidation with one shared source of truth. Students see where their thesis stands and what is still missing. Professors see only the theses in which they participate and the actions their role allows on each one. Administrative staff can schedule defenses knowing that rooms and jury members are free and that all prerequisites have been met. Enforcing the rules in the database and the API, rather than relying on human discipline, makes inconsistent states impossible, and the status history provides the traceability that audits require.

Academic motivation. The project lets the team apply the full web-application life cycle (requirements, data modeling, secure API design, front-end development, testing, and deployment) to a real problem with a real client. It also continues the previous course's work instead of discarding it: the inherited database is preserved, corrected, and turned into a usable product.

II. Reference Framework

A. Theoretical and Technical Background

Relational data model and integrity constraints. The relational model [2] represents data as relations linked by primary and foreign keys. Beyond these keys, a relational database management system (RDBMS) can enforce domain rules declaratively through `UNIQUE` and `CHECK` constraints, and procedurally through triggers. MySQL 8 enforces `CHECK` constraints from version 8.0.16 [3]. Our design uses this to guarantee, for example, that a grade lies between 0 and 10. Rules that span several rows, such as "at most three reviewers per thesis", cannot be expressed with keys alone. These are enforced by a trigger [4] and re-validated in the application's service layer.

Schema migrations. Instead of recreating the database whenever the model changes, the schema evolves through versioned, reversible migration scripts kept under version control. We use Alembic [5] together with the SQLAlchemy object-relational mapper (ORM) [6]. The first migration reproduces the inherited schema exactly. Every correction is then applied as a later migration, so the evolution of the data model remains traceable.

RESTful web APIs. Representational State Transfer (REST) [7] is an architectural style for networked systems based on resources, uniform interfaces, and stateless client–server interaction. The back end exposes the thesis domain as REST resources (students, theses, assignments, reviews, defenses) over HTTP and JSON. It is implemented with FastAPI [8], a Python framework that validates request and response data through type annotations and generates interactive OpenAPI documentation automatically.

Authentication and role-based access control. Users authenticate with credentials and receive a signed JSON Web Token (JWT) [9], which the client sends with every request. In role-based access control (RBAC) [10], [11], permissions are assigned to roles rather than to individual users. Our design combines three global roles (administrator, professor, student) with *contextual* permissions derived from the assignment tables: the same professor may be the tutor of one thesis and a reviewer of another. Security requirements follow the OWASP API Security Top 10 [12], in particular broken object-level authorization, which here means preventing a user from reading or modifying a thesis in which they do not participate.

Single-page applications. The client is a single-page application (SPA) that runs in the browser, renders views from the data returned by the API, and communicates with it asynchronously. Separating the client from the API allows each to be developed, tested, and deployed independently.

Auditing and temporal data. Every status change of a thesis is stored as an immutable history record containing the previous status, the new status, the user responsible, and a timestamp. The record is written in the same transaction as the change itself, so the history can never diverge from the actual state.

B. Related Work

Web-based support for the thesis process has been studied for more than a decade. Early work focused on automating the sequence of process stages: Li *et al.* [13] built a thesis management information system on top of a workflow engine, which dispatches each step according to the date, the logged-in user, and that user's status. Later systems narrowed their focus to specific parts of the process, such as topic selection

and supervisor guidance [14], or the communication between student and supervisor. Shakerian *et al.* [15], for instance, adapted a learning management system to the master's thesis process and found that students and supervisors preferred it to e-mail, meetings, and social networks.

Closer to our scope, ThesisIT [16] combines a thesis portal (document collection, defense minutes, rating sheets, and comments from advisers and panel members) with automatic generation of defense schedules. Hariyanto *et al.* [17] design an information system that replaces manual monitoring through general-purpose tools with a single flow, from registration to the issuance of a provisional graduation letter, but they present a design model rather than an evaluated deployment. Usability studies such as [18] confirm that these systems are being adopted and evaluated by their users. The most complete platform we found is the open-source Thesis Management system of the Technical University of Munich [19]. It covers the full thesis life cycle with a permission matrix for students, advisors, supervisors, examiners, and administrators, presentation scheduling, feedback, and grading.

In Ecuador, Silva-Peñafiel *et al.* [20] developed, using Scrum, a system that records and tracks the graduation (*titulación*) projects of a university extension. It involves the student, the tutor, and a tribunal of readers, and it supports electronic signatures. Finally, a recent study of thesis supervision workflows [21] reports that administrators carry a large amount of hidden coordination work across separate platforms, deadlines, and submissions. This finding directly motivates an integrated platform.

Table I compares these works with the proposed system on the features this project needs. Three gaps stand out. First, none of the reviewed systems reports validating the administrative requirements that Ecuadorian regulation imposes before a defense, such as internships, community engagement, and language proficiency [1]. Second, none reports keeping a record of similarity checks. Third, none reports an auditable history of status changes. Moreover, permissions are usually tied to fixed user roles, whereas in our context the same professor plays different roles in different theses. The proposed system differs in three ways: it treats these rules as first-class, database-enforced constraints; it derives permissions from each thesis's assignments; and it builds on, and corrects, a schema already designed for the institution.

III. Proposed System

A. Methodology

The project follows an iterative, incremental methodology based on Scrum [22], adapted to a small student team and to the academic calendar. Work is organized into time-boxed sprints of two weeks. Each sprint delivers a working, deployable increment that is reviewed with the client (the coordination of the graduation process) and with the course instructor (Fig. 2). Scrum has been applied successfully to student software projects in academic settings [23], and it was also used to build the closest Ecuadorian precedent of this system [20].

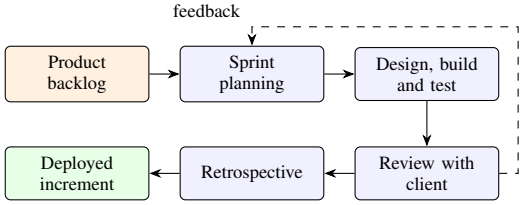


Fig. 2. Iterative development cycle. Client feedback from each sprint review is fed back into the backlog and the next sprint plan.

Phase 0: Discovery and validation. Before development begins, the team validates the open business rules with the client. These include the official list of administrative requirements, whether the system stores documents, the exact thesis status workflow, and how the final grade is computed from the jury's individual grades. All design decisions are recorded in a decision log, together with their rationale.

Data-first approach. Because the inherited database is the starting point, the first sprint establishes the migration baseline from the existing schema and applies the corrections that do not depend on the client. Changes that do depend on the client are scheduled once they are validated.

Planned increments. Table II outlines the sequence of increments. Each increment includes its API endpoints, its user interface, and its automated tests, so features are always delivered end to end.

Quality practices. All code lives in a shared Git repository, and features are developed in short-lived branches that are merged through peer-reviewed pull requests. A continuous-integration pipeline runs linting and the automated test suite on every change. A feature counts as done only when it is implemented, tested, documented in the OpenAPI specification, and demonstrated in the sprint review.

References

[1] Consejo de Educación Superior (CES), "Reglamento de régimen académico, resolución RPC-SE-08-No.023-2022, codified with reforms RPC-SE-10-No.028-2022 and RPC-SE-03-No.008-2023," Registro Oficial, Segundo Suplemento No. 287, Apr. 11, 2023, 2023, accessed: Oct. 15, 2026. [Online]. Available: https://www.gob.ec/regulaciones/reglamento-regimen-academico-0

[2] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[3] Oracle Corporation, "MySQL 8.0 reference manual: CHECK constraints," accessed: Oct. 15, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html

[4] ——, "MySQL 8.0 reference manual: Using triggers," accessed: Oct. 15, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/triggers.html

[5] SQLAlchemy authors and contributors, "Alembic documentation," accessed: Oct. 15, 2026. [Online]. Available: https://alembic.sqlalchemy.org/en/latest/

[6] ——, "SQLAlchemy 2.0 documentation," accessed: Oct. 15, 2026. [Online]. Available: https://docs.sqlalchemy.org/en/20/

[7] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[8] S. Ramírez, "FastAPI documentation," accessed: Oct. 15, 2026. [Online]. Available: https://fastapi.tiangolo.com/

TABLE I
COMPARISON OF RELATED WORK WITH THE PROPOSED SYSTEM

Work	Full workflow	Committee / jury roles	Defense scheduling	Admin. requirements	Similarity records	Status history / audit	Per-thesis permissions
Li *et al.* [13]	✓	–	–	–	–	–	–
Zhi [14]	○	–	–	–	–	–	–
Shakerian *et al.* [15]	○	–	–	–	–	–	–
Silva-Peñafiel *et al.* [20]	✓	✓	–	–	–	–	–
ThesisIT [16]	✓	✓	✓	–	–	–	–
Hariyanto *et al.* [17]	✓	–	–	–	–	–	–
TUM Thesis Mgmt. [19]	✓	✓	✓	–	–	–	○
Proposed system	✓	✓	✓	✓	✓	✓	✓

✓: reported by the authors; ○: partially covered; –: not reported in the cited source. The comparison is based only on what each source describes and does not imply that a feature is technically impossible in that system.

TABLE II
PLANNED SPRINT INCREMENTS

Sprint	Increment
0	Requirements validation with the client; decision log; repository, environments and CI pipeline; migration baseline of the inherited schema.
1	Schema corrections (keys, constraints, timestamps); authentication, users and roles.
2	Students, professors and academic periods; thesis registration; tutor and co-tutor assignment.
3	Thesis status workflow and status history; reviewer committee and review observations.
4	Similarity checks and administrative requirements; pre-defense validation gate.
5	Rooms and defense scheduling with conflict detection; jury grading.
6	Analytical reports; hardening, testing and production deployment.

[9] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[10] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[11] R. Sandhu, D. Ferraiolo, and R. Kuhn, "The NIST model for role-based access control: Towards a unified standard," in *Proc. 5th ACM Workshop on Role-Based Access Control*, 2000, pp. 47–63.

[12] OWASP Foundation, "OWASP top 10 API security risks – 2023," 2023, accessed: Oct. 15, 2026. [Online]. Available: https://api-security.owasp.org/editions/2023/en/0x11-t10

[13] C. Li, J. Shi, and W. Zhong, "An auto-management thesis program WebMIS based on workflow," *Physics Procedia*, vol. 33, pp. 892–898, 2012.

[14] H. Zhi, "The design and implementation of the topic selection and guidance system of college graduation thesis based on web," in *Proc. 2nd Int. Conf. on Information Science and Education (ICISE-IE)*. IEEE, 2021, pp. 1374–1377.

[15] S. Shakerian, S. Noori, P. Hiedarpoor, L. Shams, and M. Hosseinzadeh, "Developing a web-based learning management system (LMS) for master's thesis process in the Shahid Beheshti University of Medical Sciences," *Journal of Medical Education for Future Demands*, vol. 19, no. 4, p. e110696, 2021.

[16] M. E. B. Chio, P. M. M. Dal, J. L. Garrido, A. Baldelovar, and J. V. M. Saledaien, "ThesisIT: Web-based university thesis management portal with a defense scheduling system," *Science International (Lahore)*, vol. 34, no. 6, pp. 531–536, 2022.

[17] F. Hariyanto, T. Budiman, A. B. Yulianto, and V. Yasin, "Designing a web-based information system for monitoring final projects," *International Journal of Engineering, Science and Information Technology*, vol. 5, no. 2, pp. 142–153, 2025.

[18] D. D. Prasetya, M. J. Fuady, W. S. G. Irianto, and W. S. Pratama, "Perception analysis of web-based thesis management system using short-user experience questionnaire (UEQ-S)," in *Proc. 9th Int. Conf. on Education and Technology (ICET)*. Malang, Indonesia: IEEE, 2023, pp. 137–141.

[19] Technical University of Munich, Applied Education Technologies, "Thesis management: A centralized platform for managing the complete thesis lifecycle," https://aet.cit.tum.de/projects/edtech/thesis-management/, open-source (MIT), source at https://github.com/ls1intum/thesis-management. Accessed: Oct. 15, 2026.

[20] G. E. Silva-Peñafiel, K. P. Morales-Guamán, J. P. Chalar-Suárez, and A. F. Rodríguez-Lirio, "Implementación de un sistema mediante la metodología SCRUM del proceso de titulación en la universidad técnica de cotopaxi extensión la maná," *Polo del Conocimiento*, vol. 6, no. 10, 2021.

[21] L.-Y. Huang, D. Zyska, and I. Gurevych, "Transforming interactions in thesis supervision: An exposé-first workflow in higher education," arXiv:2608.12546, 2026, work in progress, Mensch und Computer (MuC) 2026.

[22] K. Schwaber and J. Sutherland, "The 2020 Scrum guide," Nov. 2020, accessed: Oct. 15, 2026. [Online]. Available: https://scrumguides.org/scrum-guide.html

[23] V. Mahnič, "A capstone course on agile software development using Scrum," *IEEE Transactions on Education*, vol. 55, no. 1, pp. 99–106, 2012.